

A Benchmark to Understand Communication Performance in Hybrid MPI and GPU Applications

Keira Haskins & Patrick G. Bridges
Department of Computer Science
University of New Mexico
Albuquerque, NM USA
{wasp,patrickb}@unm.edu

Scott Levy & Kurt B. Ferreira
Center for Computing Research
Sandia National Laboratories
Albuquerque, NM, USA
{sllevy,kbferre}@sandia.gov

Abstract—Analyzing MPI communication costs on extreme-scale high-performance computing systems is critical to ensuring optimal performance. Several factors such as scalability and the wide spread use of GPUs make this analysis challenging. In order to address this challenge, system and application designers need benchmarks and tools that communicate using GPU and host memory in a manner similar to production extreme-scale applications. These tools need to provide the user with sufficient options to, for example, fine-tune and analyze the costs associated with the low-level exchange of data between host and device memory. In this paper we describe the design and prototype implementation of a suite of tools designed to measure and account for data movement with GPUS. Specifically, we describe a GPU-focused communication benchmark written in Kokkos/C++ based on the FIESTA CFD and an MPI profiling library integrated with NVIDIA’s *nvprof* profiling tool. Using this combination, we demonstrate how GPU data movement overheads can vary and examine the abilities of these tools to understand performance behavior. These tools are important contributions that we believe will guide system designers and application developers in best practice data movement decisions on extreme-scale HPC systems.

Index Terms—GPU, MPI, high-performance computing

I. INTRODUCTION

Analyzing MPI communication costs on extreme-scale high-performance computing (HPC) systems is critical to ensuring optimal performance. Several factors such as scalability and the use of GPUs on modern systems make this analysis challenging. While GPUs have the potential for increased computation performance, they also have the potential to significantly impact communication costs due to expensive data movement required by some MPI applications. Managing and optimizing GPU communication and data movement in these systems is very complex due to difficult challenges related to balancing concurrency, packing data, and dealing with synchronization and memory management costs. Current benchmarks, measurement tools, and communication systems do not do an adequate job of capturing these tradeoffs and complexities and providing the user sufficient information to guide best practices and make optimal decisions.

In order to address this challenge, system and application designers need a suite of benchmarks and tools that communicate using GPU and host memory in a manner similar to real extreme-scale applications. These tools need to provide

the user with sufficient options to, for example, fine-tune and analyze the costs associated with the low-level exchange of data between host and device memory. These tools also need the ability to integrate with existing GPU profiling tools such as *nvprof* [1] and those based on CUPTI [2]. Finally, these tools need to be flexible enough to easily reorder data movement and communication operations, for example, performing all pack operations before sending or interleaving pack and send operations.

Existing tools and benchmarks in this area, for example various mini-applications including the Comb benchmark [3], are insufficient as they typically only address one application or communication pattern. Other tools, for example GPU variants of common communication benchmarks, do not fully account for the communication options or overheads associated with GPU processors found on current extreme-scale systems.

In this paper we describe the design and prototype implementation of a set of tools designed to address this problem of measuring and accounting for data movement with GPUs. This work therefore make the following contributions:

- We designed and implemented an initial GPU-focused communication benchmark written in Kokkos/C++ that exercises point-to-point communication. This code uses data structures and code taken the FIESTA CFD application [4] to exercise various data movement mechanisms.
- We integrated a prototype MPI profiling library with NVIDIA’s *nvprof* profiling tool based on known techniques to examine their ability to evaluate MPI+GPU communication overheads.
- Using the combination of this benchmark and profiling tool, we demonstrate how GPU data movement overheads can vary on modern systems and examine the abilities of these tools to understand their behavior.

II. APPROACH

Our benchmark is based on the data exchange in FIESTA [4]. In our benchmark, two processes exchange halo regions of a 4-dimensional data array stored in a `Kokkos::View`¹ in GPU memory. The halo regions in this

¹Kokkos [5] is a performance portability layer that, for our purposes, provides useful abstractions for managing data in multi-dimensional arrays across devices.

benchmark are limited to a single dimension: for example, if the 4-dimensional data array has dimensions $(n_x \times n_y \times n_z \times n_t)$ and there are n_h elements in the halo, then in the first dimension there is a halo region, with dimensions $(n_h \times n_y \times n_z \times t)$. In our benchmark, the first three dimensions— n_x , n_y , and n_z —are always equal ($n_x = n_y = n_z = n_d$) and we scale the size of the problem by changing the value of n_d .

A. Data Packing Approaches

Our benchmark can be configured to exchange halo regions using one of the following three approaches:

- **DIRECT** : passes the address of the data buffer in device memory directly to MPI. The buffer elements to send and receive are selected by creating subarrays in MPI using `MPI_Type_create_subarray`. Each subarray represents halo values sent to or received from the partner process. The MPI library uses this information to handle the process of packing the data that is sent (`MPI_Send`) and received (`MPI_Recv`).
- **CUDA-AWARE** : uses a `Kokkos::parallel_for` to copy the halo values into new (smaller) `Kokkos::Views`. The contents of these `Kokkos::Views` are then exchanged with the partner process using `MPI_Send` and `MPI_Recv`. The contents of the `Kokkos::Views` received from the partner are then copied into the combined data array.
- **COPY** : uses a `Kokkos::parallel_for` to copy the halo values into new (smaller) `Kokkos::Views`. The contents of the `Kokkos::Views` that contain the local halo values are explicitly copied to host memory before the data is sent to the partner process with `MPI_Send`. After the exchange is complete and `MPI_Recv` returns, the data received from the partner is explicitly copied back to device memory.

B. Profiling Tools

Our benchmark is designed to be integrated with profiling tools to help us develop a detailed understanding of how operations on high-level abstractions (e.g., `Kokkos::Views`) affect the low-level exchange of data between host and device memory. In our initial implementation, we have implemented support for NVIDIA’s `nvprof` profiler.

III. RESULTS

To evaluate the effectiveness of this approach, we ran the initial benchmark we developed in multiple configurations to evaluate its ability to expose different elements of MPI implementation performance and of the `nvprof`-based profiling system to capture key elements of communication system performance. Specifically, we examined the performance of different approaches to packing, unpacking, sending, and receiving on-GPU data structures for use by MPI, and how changing the layout of the memory being sent (by changing the dimension of the slide of the data structure being exchanged) impacted system performance.

We ran these tests on the Lassen [6] supercomputer at Lawrence Livermore National Laboratory. Each Lassen compute node runs a 40-core IBM Power9 CPU and 4 NVIDIA V100 (Volta) GPUs connected by Mellanox ConnectX-6 HDR 100Gb/s Infiniband network devices. For these tests, we used the system-provided MPI library, IBM Spectrum MPI version `spectrum-mpi/2020.08.19`.

A. Varying MPI Communication Strategy

Figure 1 shows the impact on the bandwidth measured on Lassen as a function of the MPI+GPU communication strategy used. This figure is limited to data collected from a halo exchange in the x -dimension in which the data being communicated is contiguous in GPU memory. Table I shows the breakdown of how GPU communication latency on the CPU and GPU in each of these cases. These results show that,

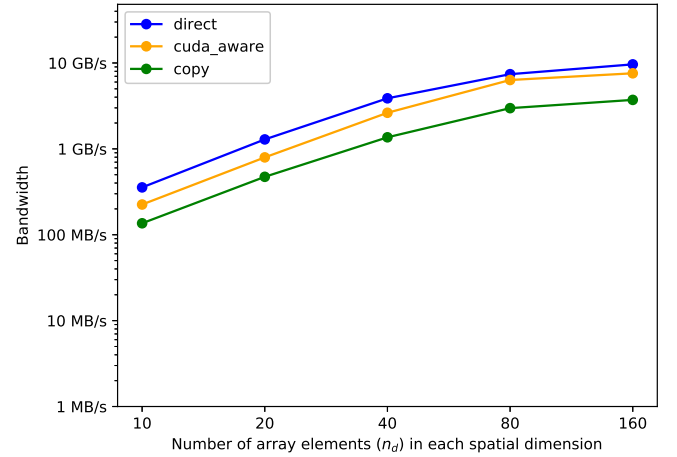


Fig. 1: Bandwidth measured on Lassen for three methods of data packing for a halo exchange in the x -dimension.

in this configuration, the direct method yields significantly higher bandwidth than the CUDA-aware or copy methods, 27% and $2.6\times$ higher for $n_d = 160$, respectively. This is due

NVProf: x Direction, $n_d = 160$			
Method	Direct	CUDA Aware	Copy
Data Pack	n/a	75.646ms	72.507ms
Device to Host Copy	n/a	n/a	242.74ms
MPI_Send	345.40ms	401.78ms	271.07ms
MPI_Recv	301.11ms	293.33ms	236.56ms
Host to Device Copy	n/a	n/a	351.95ms
Data Unpack	n/a	71.836ms	73.326ms

TABLE I: `nvprof` breakdown of benchmark time spent performing communication-related activities with different MPI+GPU communication strategies over 1000 total iterations.

to the ability to use GPUDirect communication for the large contiguous block of data being sent, completely avoiding the packing and copy costs in the other two implementations.

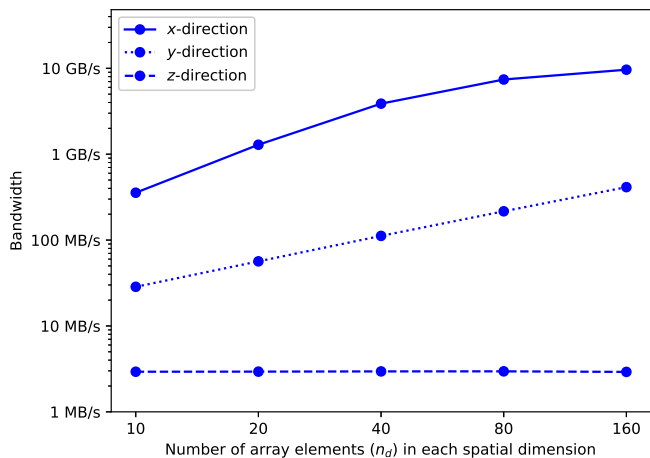


Fig. 2: Empirical bandwidth measured on Lassen for halo exchanges in three different directions for the *direct* method of data packing.

B. Varying Communication Data Layout

Figure 2 shows the impact on the bandwidth measured on Lassen as a function of the spatial dimension of the halo exchange. This figure is limited to data collected using the *direct* method of MPI+GPU communication, which relies on the implementation of MPI datatypes. Similarly, Table II shows the breakdown of how GPU communication time is spent in each of these cases. These results show that, in this configuration, while the direct method yields very high bandwidth for halo exchanges in the *x*-dimension, halo exchanges in the *y*- and *z*-dimension yield much lower bandwidth. Although we do not include the results here, the bandwidth achieved by the other two methods of GPU+MPI communication (copy and CUDA-aware) is effectively unchanged by changing the dimension that the halo communication takes place in. These results demonstrate that even relatively minor changes in communication patterns can have a significant impact on bandwidth.

NVProf: Direct Method in x, y, and z directions: $n_d = 160$			
Direction	x	y	z
Device to Host	13	160013	25600013
Host to Device	11	160011	25600011

TABLE II: Number of CUDA device memcopy GPU invocations performed by IBM Spectrum MPI over 1000 iterations when communicating halo exchanges in different dimensions.

C. Additional Discussion

In addition to the quantitative results presented above, we also evaluated the ease with which a details breakdown of MPI+GPU communication time could be determined using the provided tools. In general, the profiles provided by *nvprof* provided sufficient information to determine how time was being spent. However, due to the profiling tools' inability

to directly measure NIC activities (e.g. GPUDirect transfers, GPU waits on NIC completion events), this analysis was not always easy even for this simple communication case. Exploring additional profiling tools and better integrating these tools with both the communication system and this benchmark and new benchmarks to increase visibility into fine-grain MPI+GPU activities is an interesting potential direction for future work.

IV. CONCLUSION & FUTURE WORK

In this paper we describe the design and prototype implementation of a suite of tools designed to measure and account for data movement with GPUS. Specifically, we describe a GPU-focused communication benchmark written in Kokkos/C++ based on the FIESTA CFD and an MPI profiling library integrated with NVIDIA's *nvprof* profiling tool, using these tools to demonstrate how GPU data movement overheads can vary and examine the abilities of these tools to understand performance behavior.

For future work, we plan to focus on developing additional benchmarks that assess GPU communication with other data access patterns, examine the performance of other MPI implementation, and research better integrated GPU/NIC/CPU communication monitoring techniques. Specifically, while the combination of a GPU-oriented communication benchmark with *nvprof* provided useful performance insights, the coarse-grained information it provided and the lack of visibility provided into NIC activities (e.g. GPUDirect transfers) and GPU/NIC interactions (e.g. waits on NIC completion events) complicated performance analysis for even simple communication patterns. Research into techniques to integrate NIC, GPU, CPU, and benchmark performance profiles or traces into a fine-grained breakdown of GPU communication cost would greatly improve the ability to understand and optimize communication system performance.

ACKNOWLEDGMENT

This work was partially supported by the U.S. Department of Energy's National Nuclear Security Administration (NNSA) under the Predictive Science Academic Alliance Program (PSAAP-III), Award #DE-NA0003966.

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

REFERENCES

- [1] NVIDIA, "Profiler User's Guide," <https://docs.nvidia.com/cuda/profiler-users-guide/index.html>.
- [2] —, "CUDA Toolkit Documentation," <https://docs.nvidia.com/cuda/cupti/index.html>.

- [3] Livermore National Laboratory, “Comb v0.3.0,” <https://github.com/LLNL/Comb>.
- [4] B. Romero, “FIESTA,” <https://github.com/beromer/fiesta>.
- [5] H. C. Edwards, C. R. Trott, and D. Sunderland, “Kokkos: Enabling manycore performance portability through polymorphic memory access patterns,” *Journal of parallel and distributed computing*, vol. 74, no. 12, pp. 3202–3216, 2014.
- [6] Livermore Computing Center, “Lassen,” <https://hpc.llnl.gov/hardware/platforms/lassen>.